Control de Concurrencia Multiversión

Dr. Gerardo Rossel

FCEN, UBA

2do Cuat. 2026

Timestamping Multiversión

Definición

- Variación/Extensión de los planificadores monoversión
- Mantiene versiones históricas de los items
- Permite que las transacciones lean valores antiguos
- **Evita** aborts ocasionados por eventos **read-too-late**

Multiversión

Lectura y Escritura

Una operación de lectura de la forma $r(x)$ **lee una versión existente** de x, y una operación de escritura de la forma $w(x)$ (siempre) **crea** una nueva versión de x **o sobrescribe** una existente.

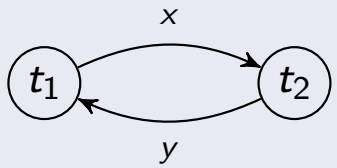
Asumimos que cada transacción escribe cada elemento de datos como máximo una vez; por lo tanto, si t_j contiene la operación $w_j(x)$, podemos denotar la versión de x creada por esta escritura como x_j.

Motivación

Monoversión: $s \notin$ CSR

$s = r_1(x)\, w_1(x)\, r_2(x)\, w_2(y)\, r_1(y)\, w_1(z)\, c_1\, c_2$

Para x, t_1 va antes que t_2; para y, t_2 va antes que $t_1 \Rightarrow$ ciclo: t_1 ve un estado inconsistente.

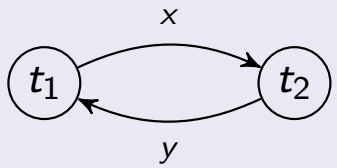


Motivación

Monoversión: $s \notin$ CSR

$s = r_1(x)\, w_1(x)\, r_2(x)\, w_2(y)\, r_1(y)\, w_1(z)\, c_1\, c_2$

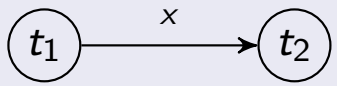
Para x, t_1 va antes que t_2; para y, t_2 va antes que t_1 $\Rightarrow$ ciclo: t_1 ve un estado inconsistente.



Multiversión: $m \in$ MVSR

$m = r_1(x_0)\, w_1(x_1)\, r_2(x_1)\, w_2(y_2)\, r_1(y_0)\, w_1(z_1)\, c_1\, c_2$

$r_1(y)$ lee la versión anterior y_0, consistente con $r_1(x_0)$ $\Rightarrow$ sin ciclo: equivalente al serial $s' = t_1\, t_2$.



Enfoque

- Cada paso de escritura w crea una nueva versión
- Cada paso de lectura r podría elegir qué versión quiere/necesita leer
- Las versiones son transparentes para la aplicación y transitorias (es decir, sujetas a recolección de basura)

Sobre Historias y Schedules

Sea $T = \{t_1, \ldots, t_n\}$ un conjunto de transacciones, donde cada $t_i \in T$ tiene la forma $t_i = (op_i, <_i)$ con op_i denotando las operaciones de t_i y $<_i$ su orden.

Una **historia** para T es un par $s = (op(s), <_s)$ tal que:

(a) $op(s) \subseteq \bigcup_{i=1}^{n} op_i \cup \bigcup_{i=1}^{n} \{a_i, c_i\}$

(b) para todo i, $1 \leq i \leq n$: $c_i \in op(s) \iff a_i \notin op(s)$

(c) $\bigcup_{i=1}^{n} <_i \subseteq <_s$

(d) para todo i, $1 \leq i \leq n$, y todo $p \in op_i$: $p <_s c_i$ o $p <_s a_i$

(e) para todo $p, q \in op(s)$ tal que al menos uno de ellos es una escritura y ambos acceden al mismo elemento de datos: $p <_s q$ o $q <_s p$

Un **schedule** es un prefijo de una historia

Función de Versión

Definición

Sea s una historia. Una **función de versión** para s es una función h, que:

- Asocia cada operación de lectura con una operación de escritura anterior del mismo ítem
- Para operaciones de escritura es la identidad

1. $h(r_i(x)) = w_j(x)$ para algún $w_j(x) <_s r_i(x)$, y $r_i(x)$ lee x_j ,
2. $h(w_i(x)) = w_i(x)$, y $w_i(x)$ escribe x_i .

Historia Multiversión

Una historia multiversión (**mv**) para las transacciones $T = \{t_1, \ldots, t_n\}$ es un par $m = (op(m), <_m)$ donde $<_m$ es un orden en $op(m)$ y

(a) $op(m) = \bigcup_{i=1}^{n} h(op(t_i))$ para alguna función de versión h,

(b) para todo $t \in T$ y todo $p, q \in op(t)$: $p <_t q \Rightarrow h(p) <_m h(q)$,

(c) si $h(r_j(x)) = r_j(x_i)$, $i \neq j$, y c_j está en m, entonces c_i está en m y $c_i <_m c_j$.

Un **schedule** multiversión (mv) es un prefijo de una historia multiversión.

Historia Multiversión



Schedule Multiversión

Una historia multiversión contiene operaciones de lecturas y escrituras versionadas para sus transacciones y el orden de las mismas respeta el orden de las transacciones individuales

Un schedule multiversión es un **schedule monoversión** si su función de versión asigna cada lectura a la última escritura precedente en el mismo elemento de datos.

Serializabilidad Multiversión

- View Serializabilidad
- Conflicto Serializabilidad



Leer de (Reads-From Relation)

Sea m un schedule multiversión, t_i y $t_j \in trans(m)$ La relación *lee-de* se define como: $RF(m) := \{(t_i, x, t_j) \mid r_j(x_i) \in op(m)\}$

Sean m y m' dos schedules multiversión tales que $trans(m') = trans(m)$ entonces m y m' son view equivalentes $(m \approx_v m')$ si $RF(m) = RF(m')$

Serializabilidad Multiversión



Multiversión view serializable

Sea m una historia multiversión, se dice que m es multiversión view serializable si existe una historia m' serial monoversión tal que $m \approx_v m'$.

MVSR es la clase de historias *view* serializables (serializables por vista) multiversión.



Decidir si una historia está en MVSR es un problema NP-Completo

Grafo de Conflictos - RF

El grafo de conflictos de una historia m denotado como $G(m)$ se construye con nodos por cada transacción de m con un eje $t_i \rightarrow t_j$ si $r_j(x_i)$ está en m
Para cualquier par de schedules multiversión, $m \approx_v m'$ **entonces** $G(m) = G(m')$.

Grafo de Conflictos - RF

El grafo de conflictos de una historia m denotado como $G(m)$ se construye con nodos por cada transacción de m con un eje $t_i \rightarrow t_j$ si $r_j(x_i)$ está en m
Para cualquier par de schedules multiversión, $m \approx_v m'$ **entonces** $G(m) = G(m')$.

$m = w_1(x_1)r_2(x_0)w_1(y_1)r_2(y_1)c_1c_2$
$m' = w_1(x_1)w_1(y_1)c_1r_2(x_1)r_2(y_0)c_2$



$G(m) = G(m')$ pero $m \not\approx_v m'$

Grafo de Serialización Multiversión (MVSG)

Definición: Un *orden de versiones* para un ítem de datos x, denotado $\ll_x$, es un orden total entre todas las versiones de x.

Un *orden de versiones* para un plan multiversión m es la unión de los órdenes de versiones para todos los ítems escritos en m.

Dado un plan m y un orden de versiones $\ll$, el *Grafo de Serialización Multiversión*, $MVSG(m, \ll)$, es un grafo dirigido donde:

- Los nodos son las transacciones commiteadas de m (su proyección commiteada, $CP(m)$)
- Las aristas se definen por las reglas de conflicto y de orden de versiones

Aristas en $MVSG(m, \ll)$

$MVSG(m, \ll)$ contiene las siguientes aristas:

(i) Todas las aristas del grafo de conflictos $G(m)$. (Para $r_k(x_j) \in op(m)$, agregar arista $t_j \rightarrow t_k$)

(ii) Para $r_k(x_j), w_i(x_i) \in CP(m)$, con k, i, j distintos: Si $x_i \ll x_j$, entonces agregar arista $t_i \rightarrow t_j$.

(iii) Para $r_k(x_j), w_i(x_i) \in CP(m)$, con k, i, j distintos: Si $x_j \ll x_i$, entonces agregar arista $t_k \rightarrow t_i$.

Intuición de las reglas (ii) y (iii)

En la serial monoversión equivalente, t_k debe leer la *última* escritura anterior a su lectura, es decir x_j. Entonces otra escritura $w_i(x_i)$ no puede quedar *entre* $w_j(x_j)$ y $r_k(x_j)$. Hay dos salidas:

(ii) $\ldots\ w_i(x)\ \ldots\ w_j(x)\ \ldots\ r_k(x)$ ✓ $t_i \rightarrow t_j$

Intuición de las reglas (ii) y (iii)

En la serial monoversión equivalente, t_k debe leer la *última* escritura anterior a su lectura, es decir x_j. Entonces otra escritura $w_i(x_i)$ no puede quedar *entre* $w_j(x_j)$ y $r_k(x_j)$. Hay dos salidas:

(ii)	$\ldots\ w_i(x)\ \ldots\ w_j(x)\ \ldots\ r_k(x)$	✓	$t_i \rightarrow t_j$
(iii)	$\ldots\ w_j(x)\ \ldots\ r_k(x)\ \ldots\ w_i(x)$	✓	$t_k \rightarrow t_i$

Intuición de las reglas (ii) y (iii)

En la serial monoversión equivalente, t_k debe leer la *última* escritura anterior a su lectura, es decir x_j. Entonces otra escritura $w_i(x_i)$ no puede quedar *entre* $w_j(x_j)$ y $r_k(x_j)$. Hay dos salidas:

(ii)	$\ldots\ w_i(x)\ \ldots\ w_j(x)\ \ldots\ r_k(x)$	✓	$t_i \rightarrow t_j$
(iii)	$\ldots\ w_j(x)\ \ldots\ r_k(x)\ \ldots\ w_i(x)$	✓	$t_k \rightarrow t_i$
—	$\ldots\ w_j(x)\ \ldots\ w_i(x)\ \ldots\ r_k(x)$	✗	k leería x_i, no x_j

Intuición de las reglas (ii) y (iii)

En la serial monoversión equivalente, t_k debe leer la *última* escritura anterior a su lectura, es decir x_j. Entonces otra escritura $w_i(x_i)$ no puede quedar *entre* $w_j(x_j)$ y $r_k(x_j)$. Hay dos salidas:

(ii)	$\ldots\ w_i(x)\ \ldots\ w_j(x)\ \ldots\ r_k(x)$	✓	$t_i \rightarrow t_j$
(iii)	$\ldots\ w_j(x)\ \ldots\ r_k(x)\ \ldots\ w_i(x)$	✓	$t_k \rightarrow t_i$
—	$\ldots\ w_j(x)\ \ldots\ w_i(x)\ \ldots\ r_k(x)$	✗	k leería x_i, no x_j

El orden de versiones $\ll$ **no** es el orden de ejecución: es el orden en que aparecerían las escrituras en la serial equivalente.

Una historia multiversión m pertenece a $MVSR$ **si y solo si** existe un orden de versiones $\ll$ tal que $MVSG(m, \ll)$ sea **acíclico**.

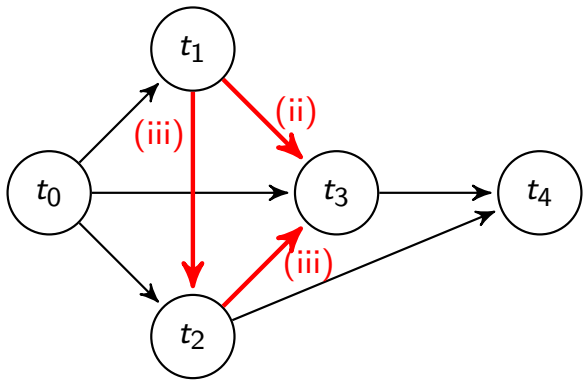
Ejemplo MVSG

Historia multiversión (schedule m):

$$m = w_0(x_0)\ w_0(y_0)\ w_0(z_0)\ c_0\ r_1(x_0)\ r_2(x_0)\ r_2(z_0)\ r_3(z_0)$$
$$w_1(y_1)\ w_2(x_2)\ w_3(y_3)\ w_3(z_3)\ c_1\ c_2\ c_3\ r_4(x_2)\ r_4(y_3)\ r_4(z_3)\ c_4$$

Orden de versiones:

$$x_0 \ll x_2, \quad y_0 \ll y_1 \ll y_3, \quad z_0 \ll z_3$$



(t_1, t_2) : $r_1(x_0), w_2(x_2) \in op(m)$, $x_0 \ll x_2$

(t_1, t_3) : $w_1(y_1), r_4(y_3) \in op(m)$, $y_1 \ll y_3$

(t_2, t_3) : $r_2(z_0), w_3(z_3) \in op(m)$, $z_0 \ll z_3$

Leyenda
$\rightarrow$ lee-de, $G(m)$ (i)
$\rightarrow$ orden de versiones (ii), (iii)

MVSG acíclico $\Rightarrow m \in$ MVSR
Serial equivalente: $t_0\ t_1\ t_2\ t_3\ t_4$

MCSR



MCSR

Vamos a trabajar con una subclase de MVSR cuya pertenencia puede determinarse en tiempo polinómico.

Conflicto Serializable



Multiversión conflicto serializable

Un conflicto multiversión en un schedule multiversión m es un par de pasos $r_i(x_j)$ y $w_k(x_k)$ tales que $r_i(x_j) <_m w_k(x_k)$

MCSR: ¿qué pares de operaciones son conflicto?

Dos operaciones p y q están en conflicto si **permutarlas** puede hacer que un schedule correcto deje de serlo.

Par (mismo ítem)	¿Conflicto?	Por qué
write–write	✗	Crean versiones distintas; cada lectura elige la que necesita.

MCSR: ¿qué pares de operaciones son conflicto?

Dos operaciones p y q están en conflicto si **permutarlas** puede hacer que un schedule correcto deje de serlo.

Par (mismo ítem)	¿Conflicto?	Por qué
write–write	✗	Crean versiones distintas; cada lectura elige la que necesita.
write–read	✗	Pasar la lectura *antes* de la escritura solo le quita opciones de versión: si así es MVSR, el original también.

MCSR: ¿qué pares de operaciones son conflicto?

Dos operaciones p y q están en conflicto si **permutarlas** puede hacer que un schedule correcto deje de serlo.

Par (mismo ítem)	¿Conflicto?	Por qué
write–write	✗	Crean versiones distintas; cada lectura elige la que necesita.
write–read	✗	Pasar la lectura *antes* de la escritura solo le quita opciones de versión: si así es MVSR, el original también.
read–write	✓	Pasar la lectura *después* de la escritura le da una opción más: que así sea MVSR no garantiza el original.

MCSR: ¿qué pares de operaciones son conflicto?

Dos operaciones p y q están en conflicto si **permutarlas** puede hacer que un schedule correcto deje de serlo.

Par (mismo ítem)	¿Conflicto?	Por qué
write–write	✗	Crean versiones distintas; cada lectura elige la que necesita.
write–read	✗	Pasar la lectura *antes* de la escritura solo le quita opciones de versión: si así es MVSR, el original también.
read–write	✓	Pasar la lectura *después* de la escritura le da una opción más: que así sea MVSR no garantiza el original.

Solo los pares *read–write* son conflictos multiversión, aunque lean y escriban versiones distintas.

Conmutatividad



Pasos de transformación

Un paso de transformación intercambia el orden de dos pasos adyacentes (es decir, pasos p, q con $p < q$ tal que $o < p$ o $q < o$ para todos los demás pasos o) pero sin invertir el orden de un par de conflicto multiversión (es decir, rw).



Reducibilidad multiversión

Una historia multiversión es *multiversión reducible* si puede **transformarse** en una *historia serial monoversión* mediante una secuencia de pasos de transformación.

Multiversión conflicto serializable



Una historia multiversión m es multiversión conflicto serializable si hay una historia monoversión serial para el mismo conjunto de transacciones en el que todos los pares de operaciones en conflicto multiversión ocurren en el mismo orden que en m.

MCSR denota la clase de todas las historias multiversión conflicto serializables.



Teorema

Una historia multiversión es multiversión reducible **si y solo si** es multiversión conflicto serializable,

$$MCSR \subset MVSR$$

Grafo de conflicto multiversión

Sea m un schedule de multiversión. El *grafo de conflicto multiversión* de m es un grafo que tiene las transacciones de m como sus nodos y una arista de t_i a t_k si existen pasos $r_i(x_j)$ y $w_k(x_k)$ para el mismo elemento de datos x en m tal que $r_i(x_j) <_m w_k(x_k)$.

Una historia multiversión es MCSR si y solo si su grafo de conflictos de multiversión es acíclico

$m \in MCSR \iff$ multiversión reducible $\iff$ grafo conflicto multiversión es acíclico.

Atención



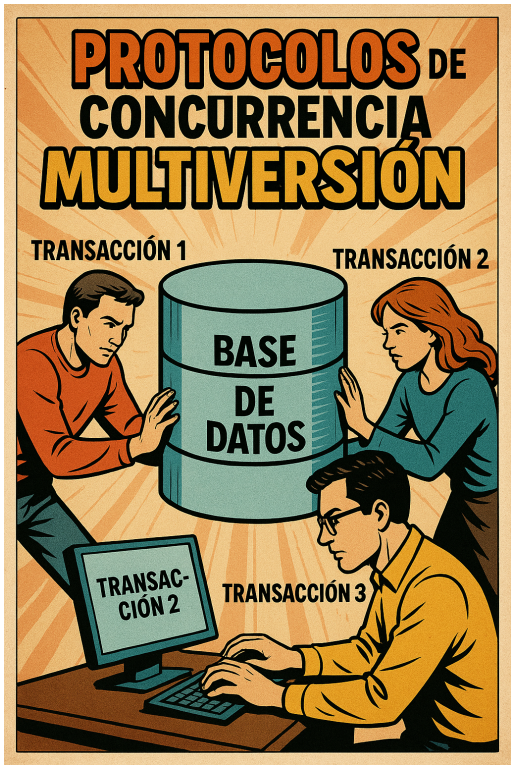
La historia monoversión serial equivalente para un grafo de conflictos multiversión acíclico no se puede derivar simplemente ordenando topológicamente el grafo



La aciclicidad del grafo es una condición suficiente y necesaria para la existencia de una secuencia de pasos de transformación (en el sentido de la definición de reducibilidad multiversión) que produzca una historia monoversión serial

Protocolos

- MVTO multiversion timestamp ordering
- MV2PL multiversion two-phase locking
  - 2V2PL two version two-phase locking
- ROMV Read Only Multiversion



Multiversion Timestamp Ordering

Cada transacción t_i recibe un timestamp $ts(t_i)$ **al comenzar**, y cada versión lleva el timestamp de la transacción que la creó.

1. Una operación $r_i(x)$ se transforma en una operación multiversión $r_i(x_k)$ donde x_k es la versión de x que tiene el timestamp mas grande menor o igual que $ts(t_i)$ y que fue escrita por $t_k, k \neq i$ *(las lecturas nunca se rechazan)*
2. Una operación $w_i(x)$ se procesa de la siguiente manera
   - Si una operación $r_j(x_k)$ tal que $ts(t_k) < ts(t_i) < ts(t_j)$ ya existe en el schedule entonces $w_i(x)$ es rechazada y t_i es abortada. Se produce un *write too late*: t_j ya leyó una versión anterior y debería haber visto x_i.
   - en otro caso $w_i(x)$ se transforma en $w_i(x_i)$ y es ejecutada.
3. Un commit c_i se retrasa hasta que los commit c_j de todas las transacciones t_j que han escrito nuevas versiones de los elementos de datos leídos por t_i hayan sido ejecutados.

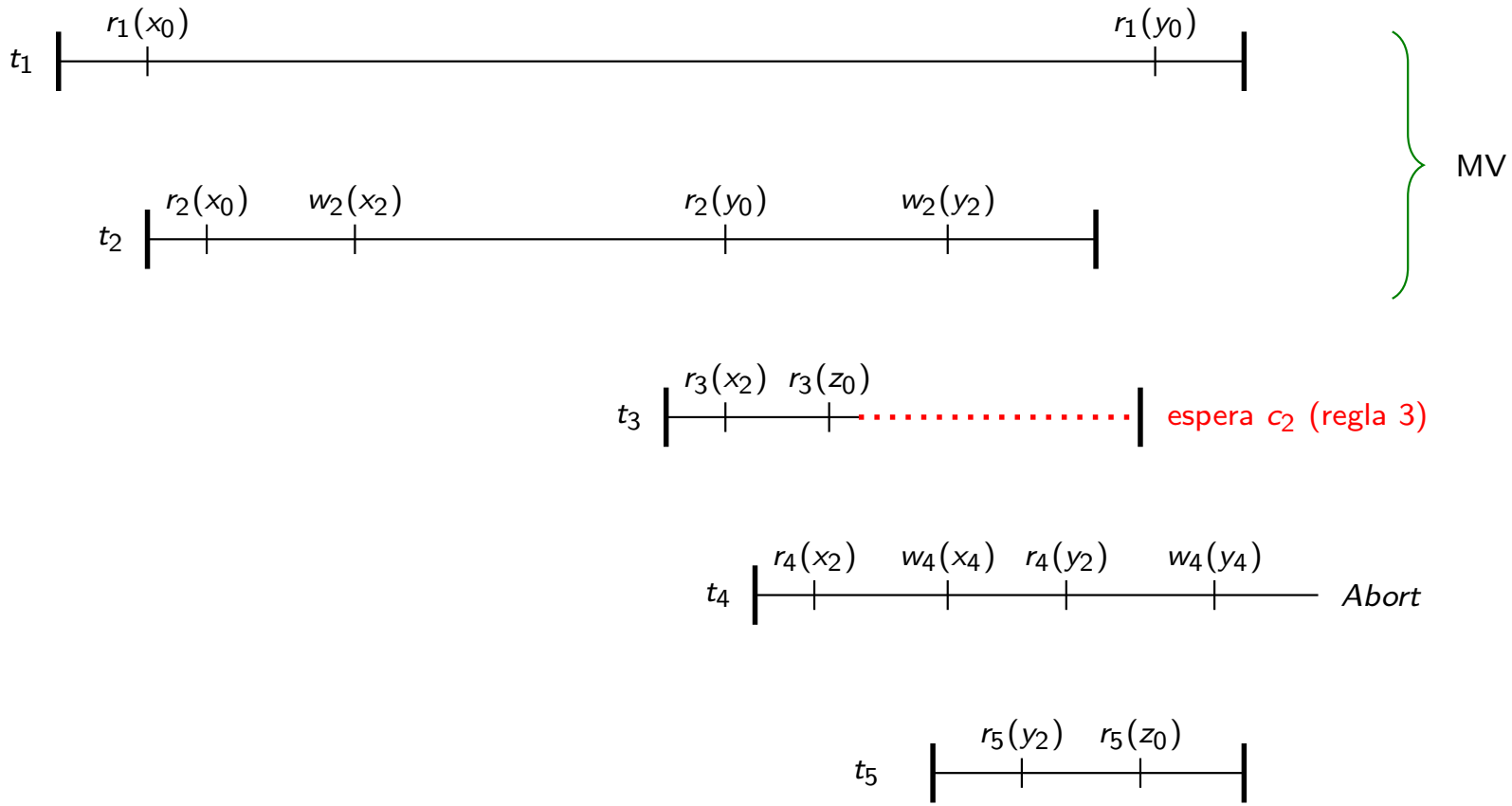
   Evita lecturas sucias: si t_j abortara, t_i (que leyó su versión) también tendría que abortar.

Ejemplo MVTO

$r_1(x); r_2(x); w_2(x); r_3(x); r_2(y); r_4(x); r_3(z); w_4(x); w_2(y); r_5(y); r_4(y); r_5(z); r_1(y); w_4(y)$
con $ts(t_1) < ts(t_2) < ts(t_3) < ts(t_4) < ts(t_5)$

Ejemplo MVTO

$r_1(x); r_2(x); w_2(x); r_3(x); r_2(y); r_4(x); r_3(z); w_4(x); w_2(y); r_5(y); r_4(y); r_5(z); r_1(y); w_4(y)$
con $ts(t_1) < ts(t_2) < ts(t_3) < ts(t_4) < ts(t_5)$



t_4 aborta: $r_5(y_2)$ ya fue ejecutada y $ts(t_2) < ts(t_4) < ts(t_5)$
$\Rightarrow$ *write too late* en $w_4(y)$

Multiversion two-phase Locking



MV2PL

El MV2PL es un protocolo basado en locking que usa *strong strict two-phase locking* o **2PL riguroso**.

Para cada elemento de datos x se distinguen:

- *versiones commiteadas*, que han sido escritas por transacciones que ya están commiteadas,
- *versión actual*: la versión commiteada de x escrita por la última transacción commiteada;
- *versiones no commiteadas*: todas las versiones restantes (creadas por transacciones que todavía están activas).

Las escrituras de un mismo ítem se serializan: una $w(x)$ que no es el paso final espera hasta que no haya otra versión no commiteada de x (regla 1b).

Multiversion two-phase Locking

Paso final de t_i: su último paso de datos antes del commit, o el commit mismo.

1. Si el paso no es el final dentro de una transacción:
   1. Una lectura $r(x)$ se ejecuta de inmediato, asignándole la versión actual del elemento de datos solicitado, es decir, la versión commiteada más recientemente (pero no cualquier otra, previamente commiteada), o asignándole una versión no commiteada de x
   2. Una escritura $w(x)$ se ejecuta solamente cuando la transacción que ha escrito x por última vez terminó, es decir, no hay otra versión no commiteada de x (se liberó su lock de escritura sobre x)
2. Si es el paso final de la transacción t_i, esta se retrasa hasta que commitean las siguientes transacciones
   1. todas aquellas transacciones t_j que hayan leído la versión actual de un elemento de datos que t_i escribió (la que t_i va a reemplazar)
   2. Todas aquellas t_j de las que t_i ha leído alguna versión.

Ejemplo MV2PL

Entrada: $s = r_1(x)\, w_1(x)\, r_2(x)\, w_2(y)\, r_1(y)\, w_2(x)\, c_2\, w_1(y)\, c_1$

1. $r_1(x)$ se asigna a la versión actual x_0 y se ejecuta: $r_1(x_0)$
2. $w_1(x)$ se ejecuta: no hay otra versión no commiteada de x: $w_1(x_1)$
3. $r_2(x)$ se asigna a la versión no commiteada x_1: $r_2(x_1)$
4. $w_2(y)$ se ejecuta: $w_2(y_2)$
5. $r_1(y)$ se asigna a la versión actual y_0: $r_1(y_0)$
6. $w_2(x)$ es el paso final de t_2: **espera**, porque (a) t_1 leyó la versión actual y_0 de y, que t_2 sobrescribe, y (b) t_2 leyó x_1 de t_1.
7. $w_1(y)$ es el paso final de t_1: se ejecuta, porque (a) t_2 no leyó la versión actual de ningún ítem escrito por t_1 (las versiones actuales son x_0, y_0), y (b) t_1 no leyó ninguna versión de t_2: $w_1(y_1)$
8. Finalmente se ejecuta $w_2(x)$: $w_2(x_2)$

2V2PL protocol

El protocolo 2V2PL (bloqueo de dos versiones) mantiene como máximo dos versiones de cualquier elemento de datos en cada momento.

- Supongamos que t_i escribe el elemento de datos x, pero aún no está *comiteado*, las dos versiones de x son su imagen anterior y su imagen posterior.
- Tan pronto como t_i se comitea, la imagen anterior puede ser eliminada ya que la nueva versión de x ahora es estable, y las versiones antiguas ya no son necesarias ni se mantienen.
- Como a lo sumo hay una versión no commiteada por ítem (lo garantiza el write lock), nunca hay más de dos versiones: la actual y la no commiteada.

En 2V2PL las operaciones de lectura están restringidas a leer solo las versiones actuales, es decir, la última versión comiteada

2V2PL protocol - Locks

- *rl* **read lock**: que se establece antes de una operación de lectura $r(x)$ respecto de la versión actual de x
- *wl* **write lock**; que se establece antes de una operación de escritura $w(x)$ para escribir una versión no commiteada de x.
- *cl* **commit o certify lock**:se establece un $cl(x)$ antes de la ejecución del paso final de una transacción en cada elemento de datos x que esta transacción **ha escrito**.



2V2PL

Los locks son sobre ítems (no sobre versiones) y los unlock obedecen al protocolo **2PL**. Como *wl* es incompatible con *wl*, hay a lo sumo una versión no commiteada por ítem. Matriz de compatibilidad:

	rl(x)	*wl(x)*	*cl(x)*
rl(x)	+	+	−
wl(x)	+	−	−
cl(x)	−	−	−

Ejercicio

Suponga el siguiente schedule que se quiere ejecutar sobre un motor que soporta **2V2PL**.

$$r_1(x); w_2(y); r_1(y); w_1(x); c_1; r_3(y); r_3(z); w_3(z); w_2(x); c_2; w_4(z); c_4; c_3$$

2V2PL

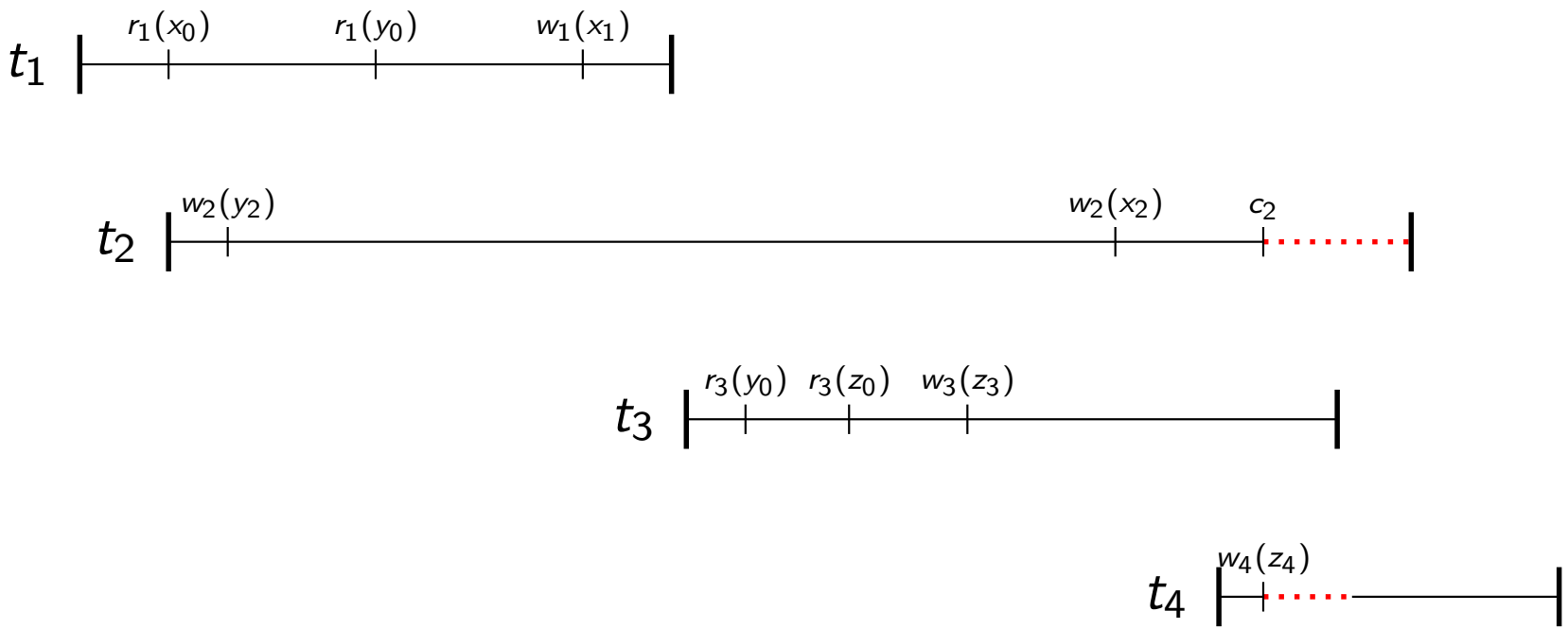
Indicar que ocurre en cada paso y en que lugar las operaciones deben ser demoradas. Escriba el *schedule* final indicando *locks y unlocks*.

Solución

$r_1(x); w_2(y); r_1(y); w_1(x); c_1; r_3(y); r_3(z); w_3(z); w_2(x); c_2; w_4(z); c_4; c_3$

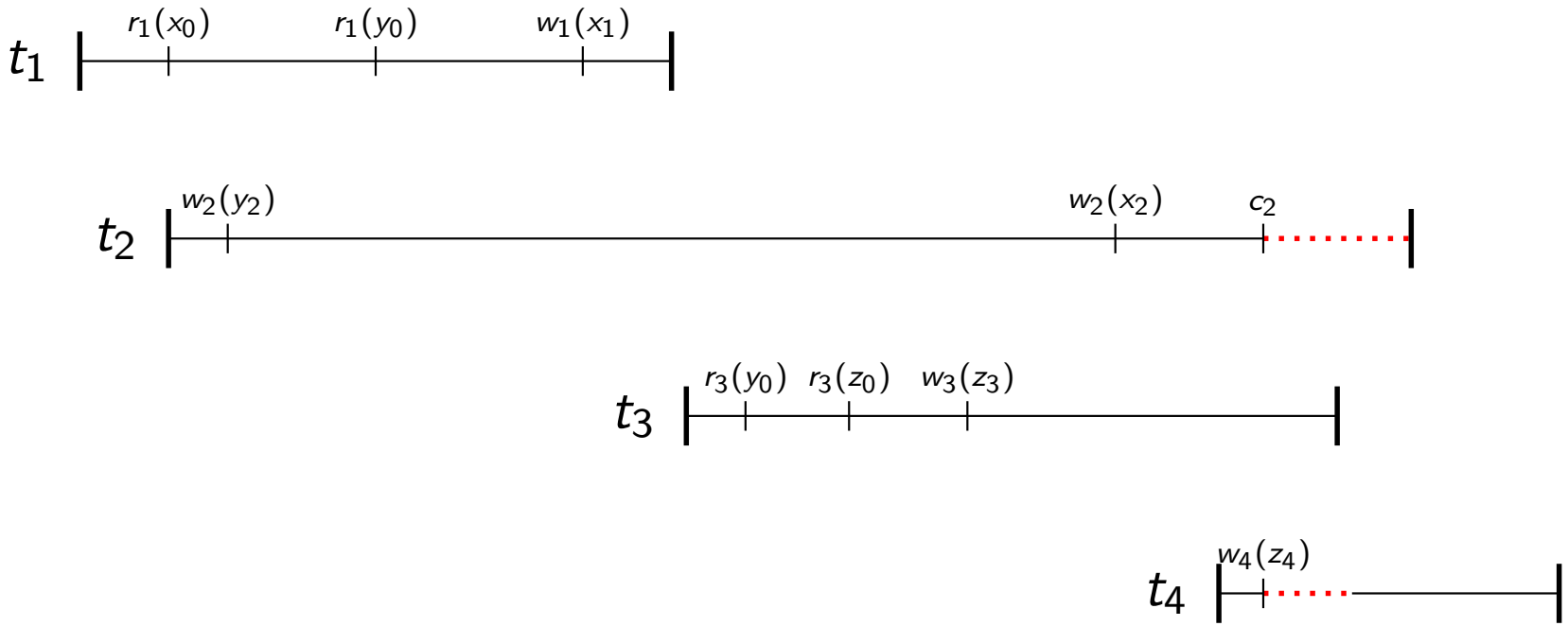
Solución

$r_1(x); w_2(y); r_1(y); w_1(x); c_1; r_3(y); r_3(z); w_3(z); w_2(x); c_2; w_4(z); c_4; c_3$



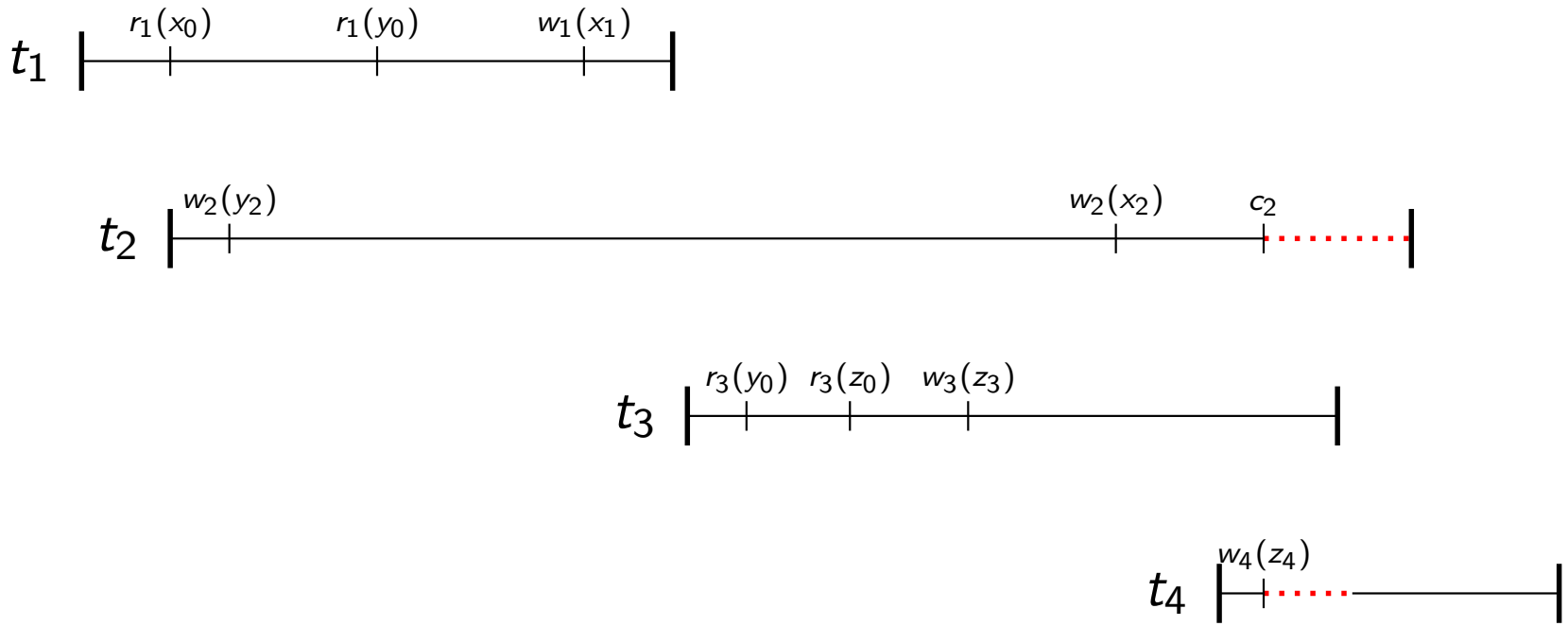
Solución

$r_1(x); w_2(y); r_1(y); w_1(x); c_1; r_3(y); r_3(z); w_3(z); w_2(x); c_2; w_4(z); c_4; c_3$

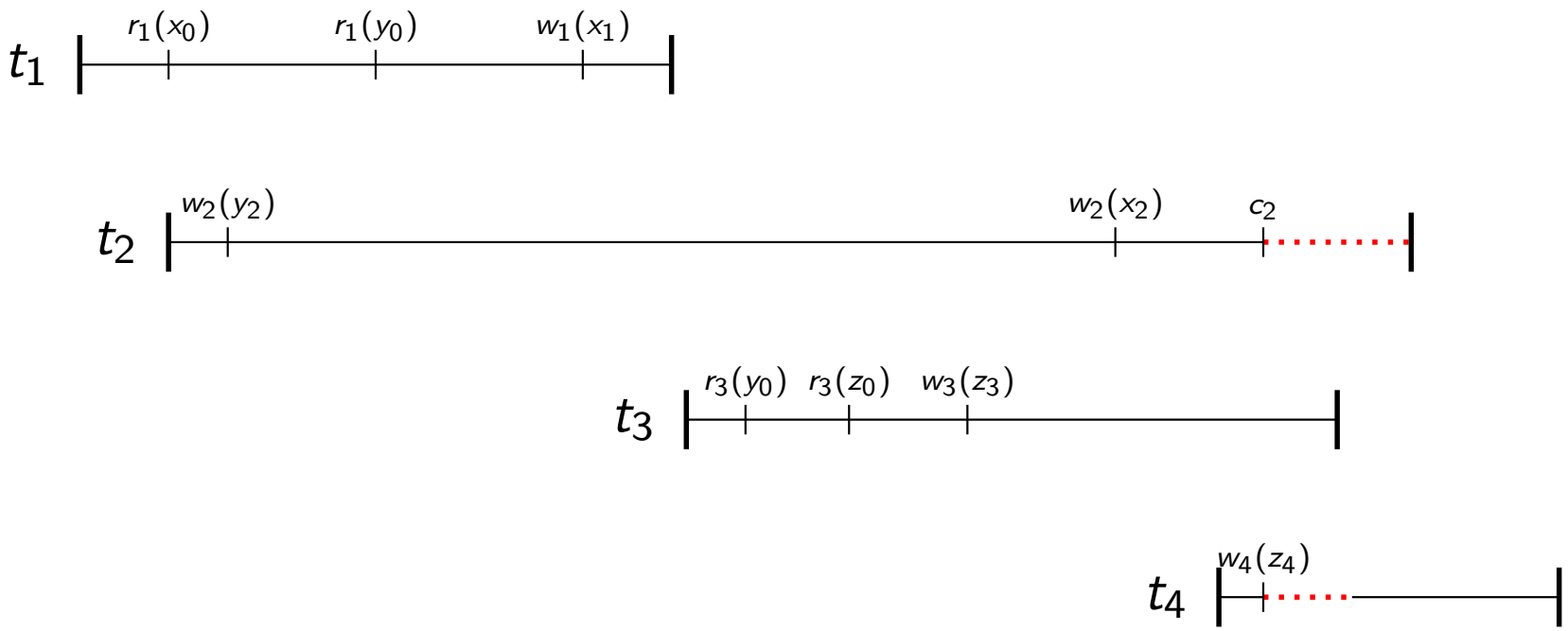


Solución

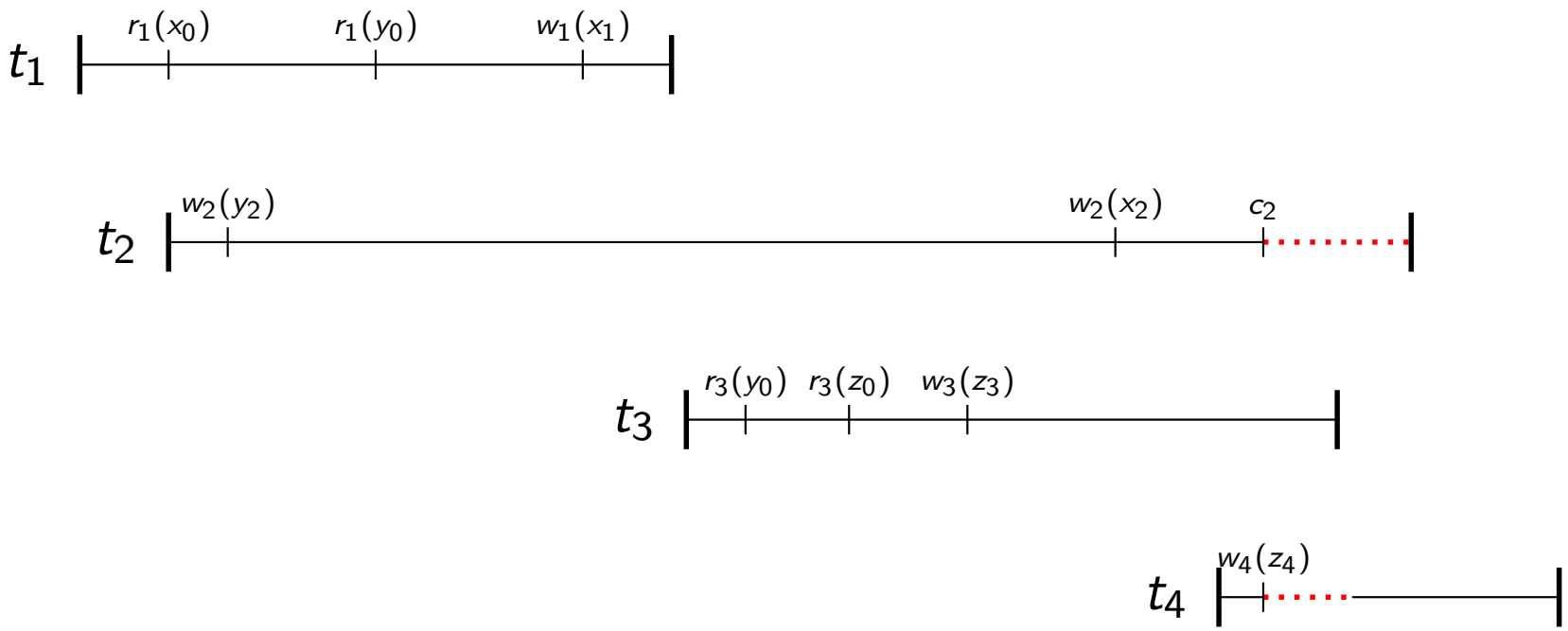
$r_1(x); w_2(y); r_1(y); w_1(x); c_1; r_3(y); r_3(z); w_3(z); w_2(x); c_2; w_4(z); c_4; c_3$



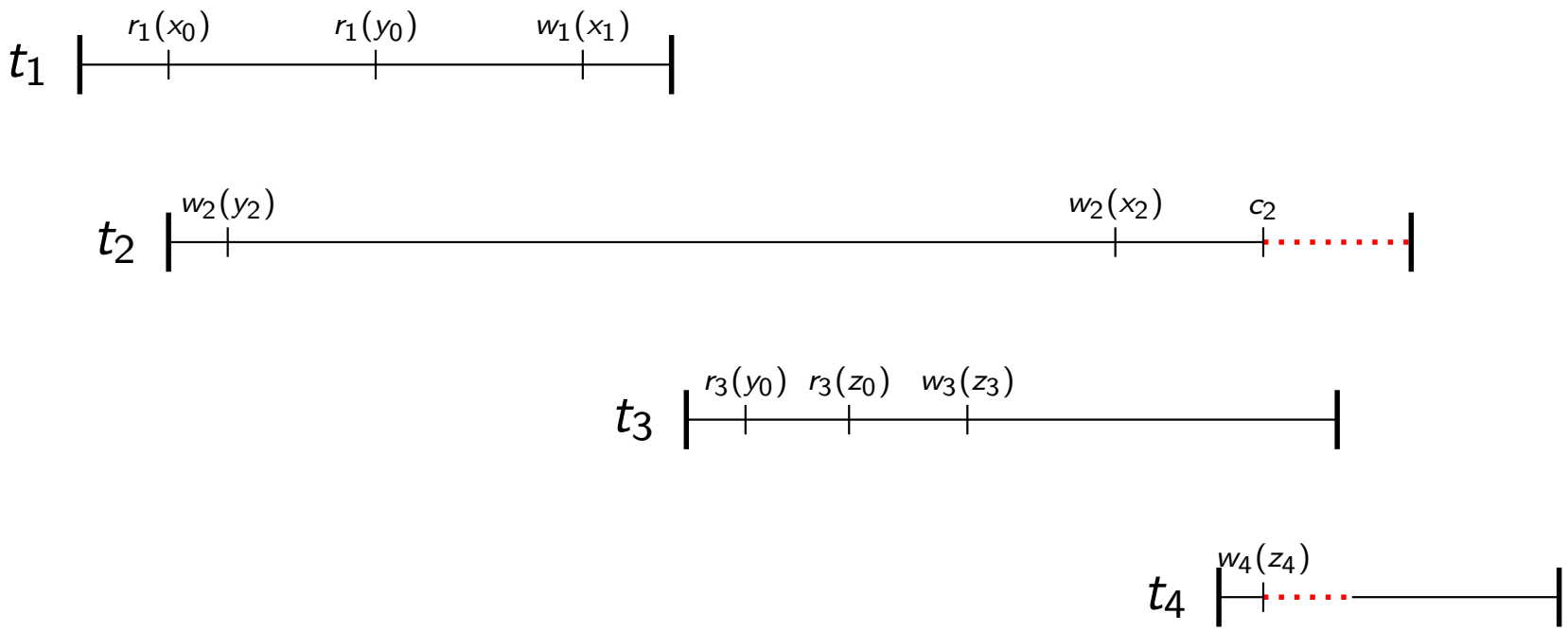
$rl_1(x); r_1(x_0); wl_2(y); w_2(y_2); rl_1(y); r_1(y_0); wl_1(x); w_1(x_1); cl_1(x); ul_1;$
$c_1; rl_3(y); r_3(y_0); rl_3(z); r_3(z_0); wl_3(z); w_3(z_3); wl_2(x); w_2(x_2);$
$cl_2(x); cl_3(z); ul_3; c_3; cl_2(y); ul_2; c_2; wl_4(z); w_4(z_4); cl_4(z); ul_4; c_4$



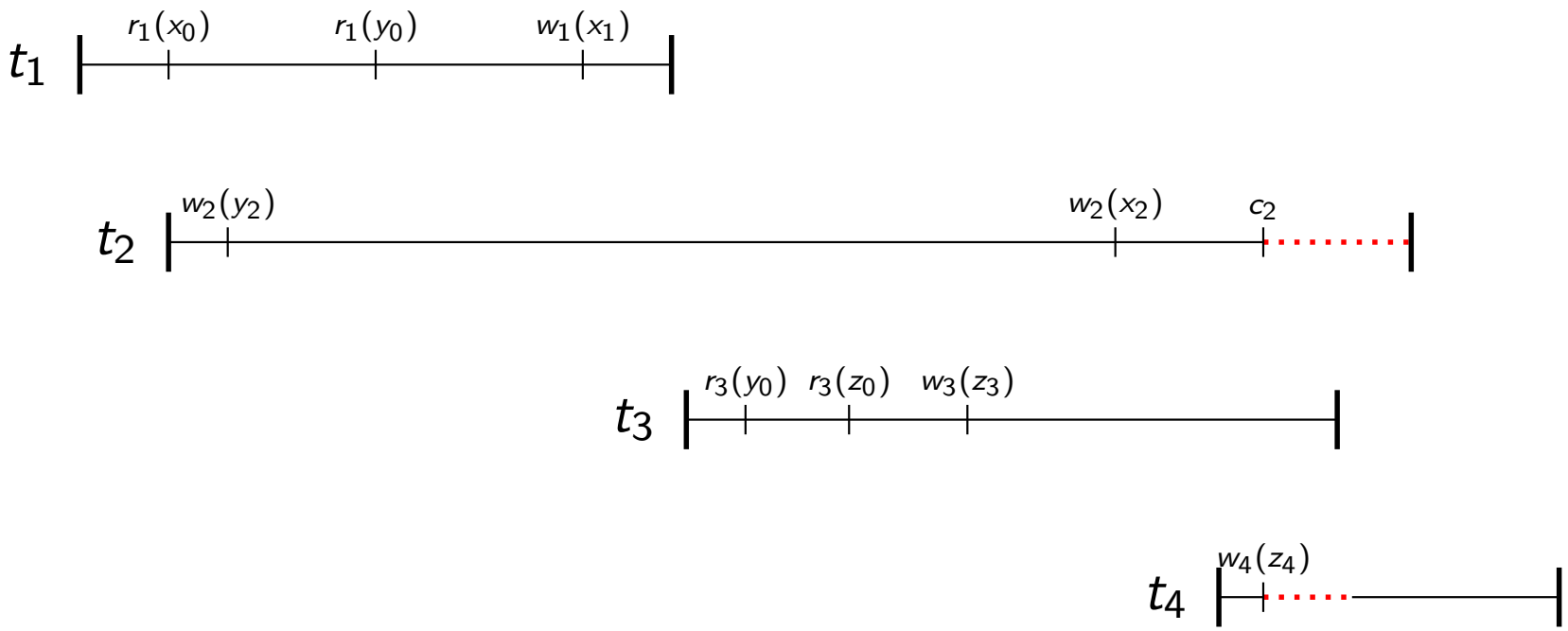
X	Y	Z
x_0	y_0	z_0



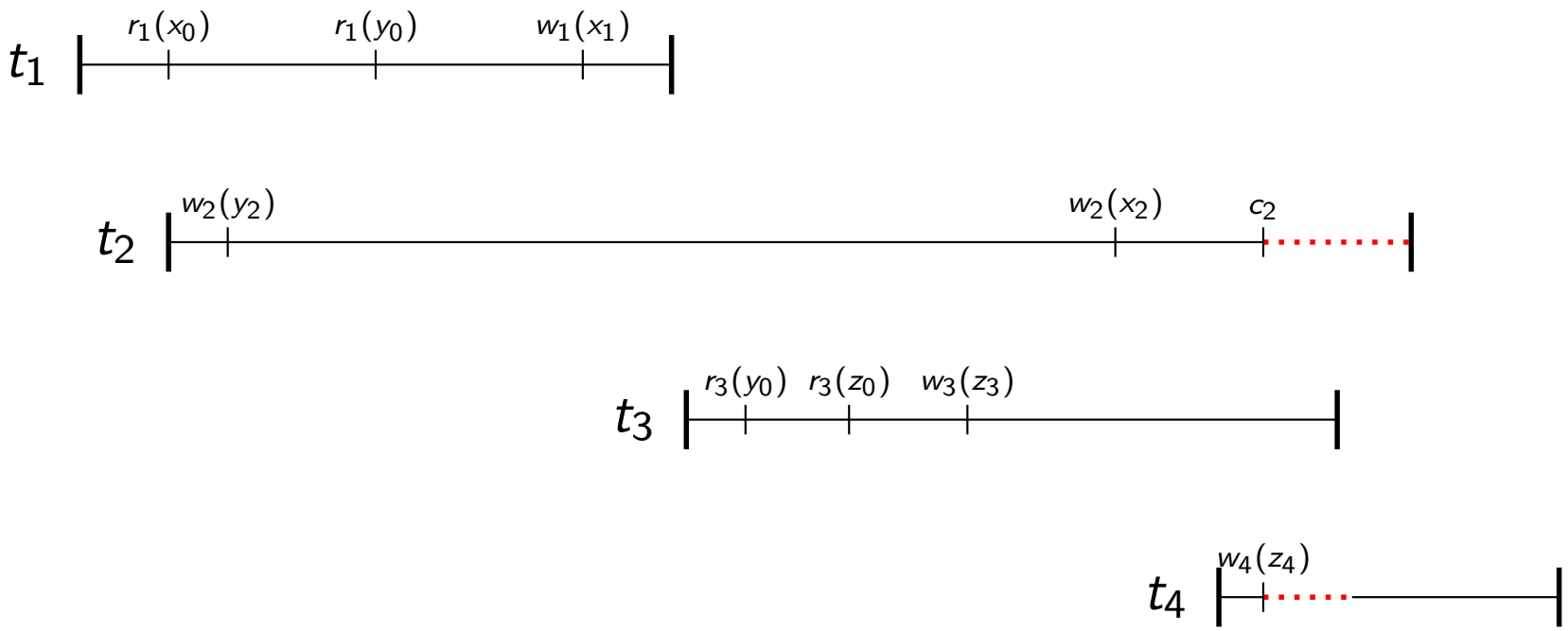
X	Y	Z
x_0	y_0 y_2	z_0



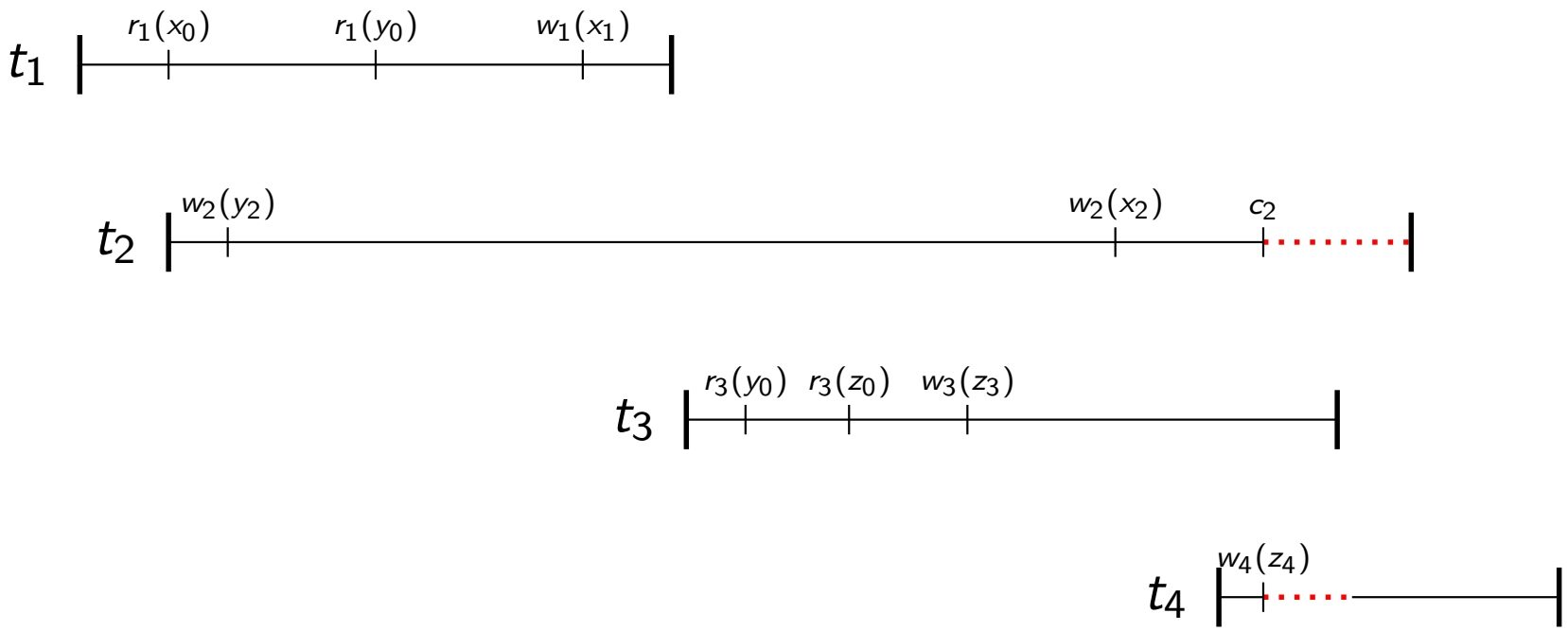
X		Y		Z
x_0	x_1	y_0	y_2	z_0



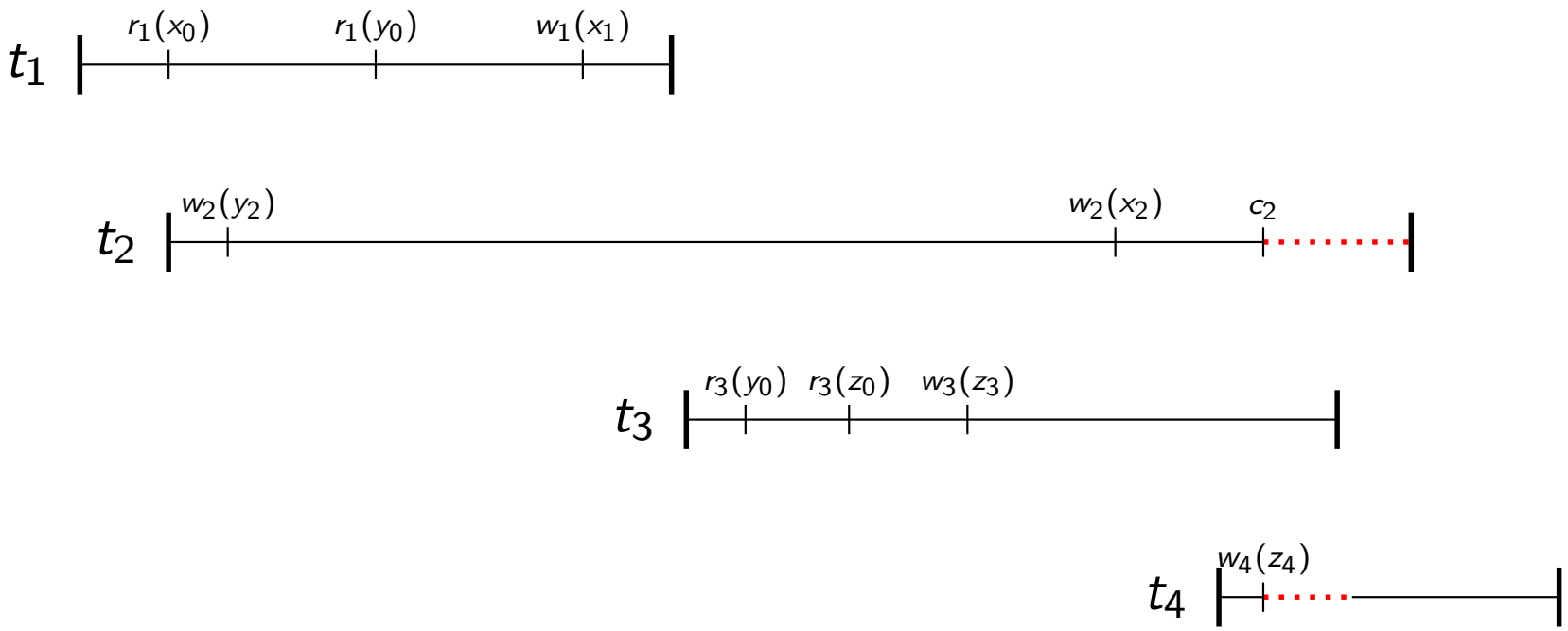
X	Y	Z
x_1	y_0 y_2	z_0



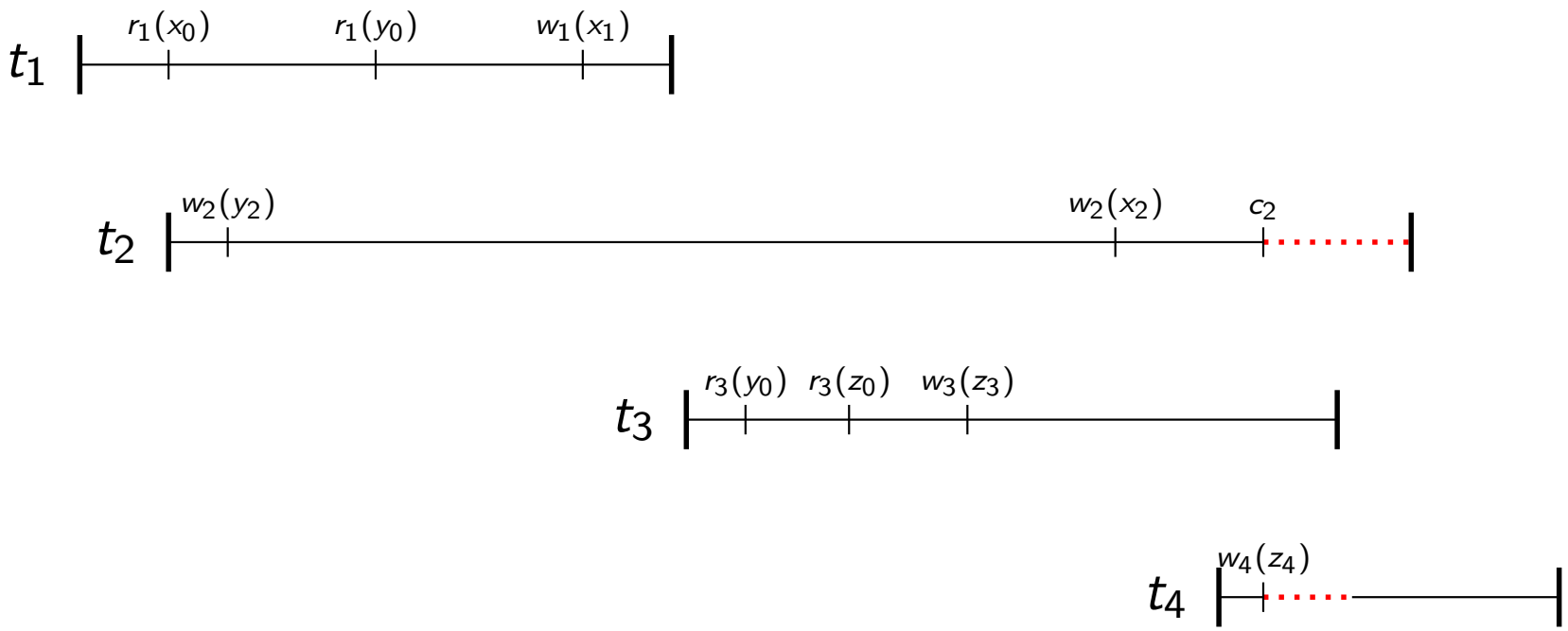
X	Y		Z	
x_1	y_0	y_2	z_0	z_3



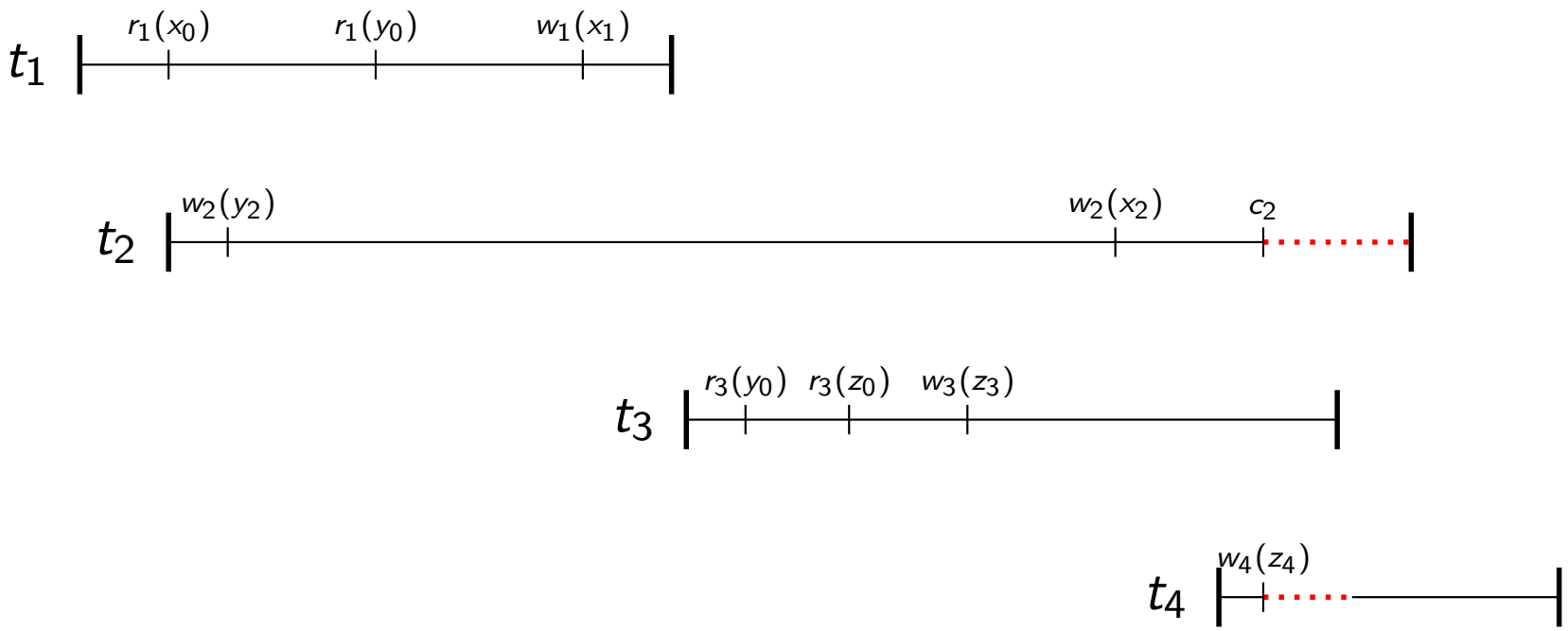
X		Y		Z	
x_1	x_2	y_0	y_2	z_0	z_3



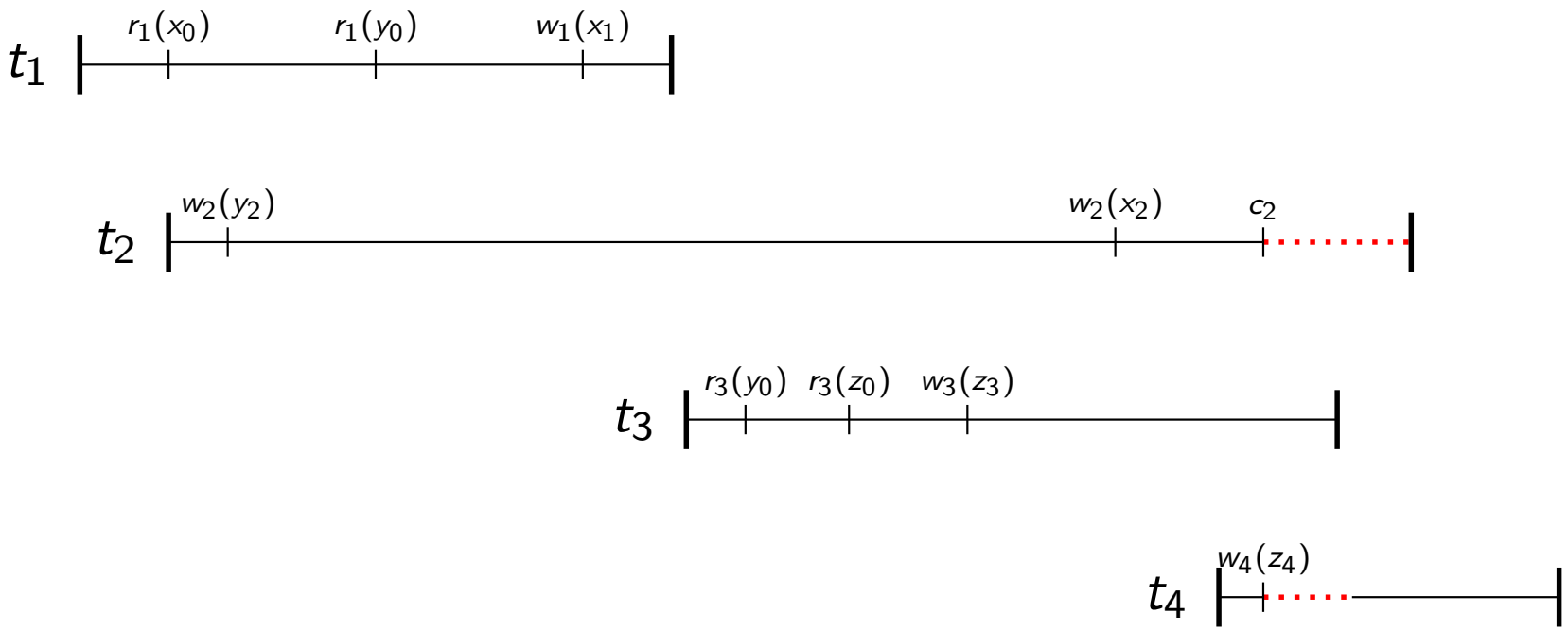
X		Y		Z
x_1	x_2	y_0	y_2	z_3



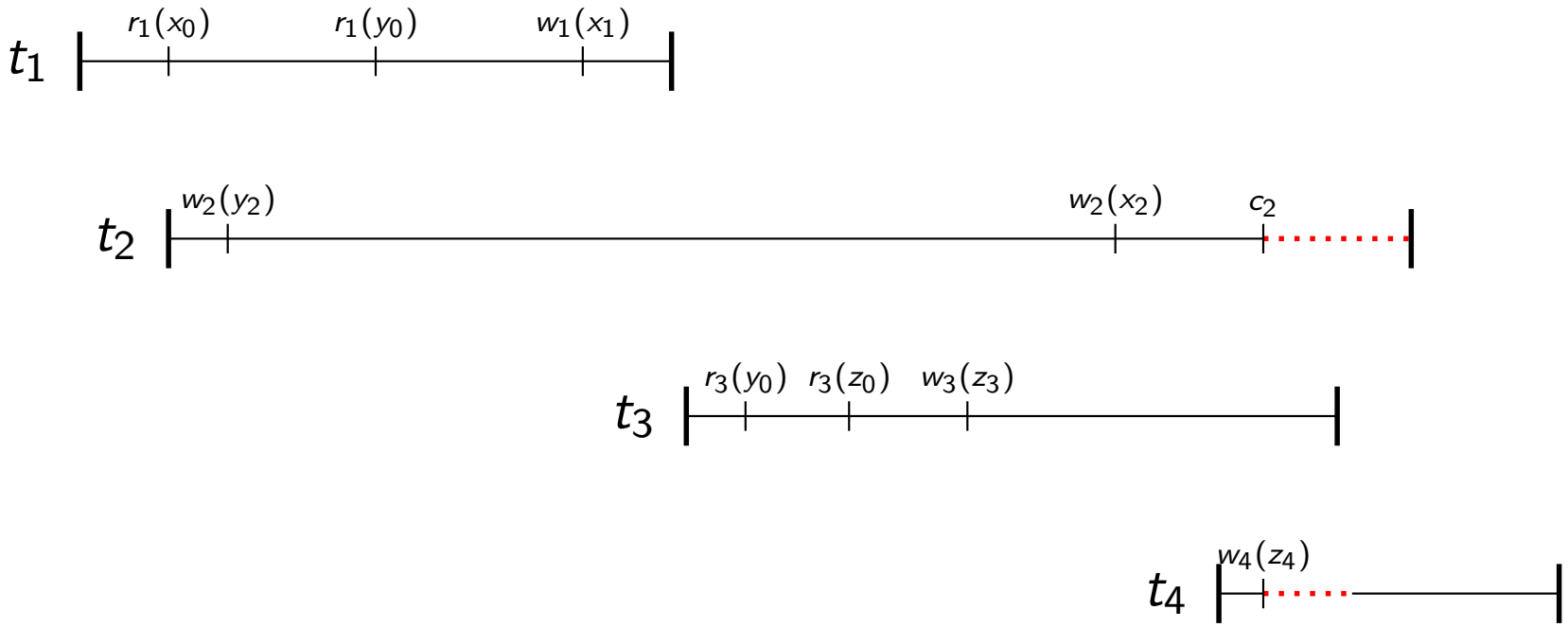
X	Y	Z
x_2	y_2	z_3



X	Y	Z	
x_2	y_2	z_3	z_4



X	Y	Z
x_2	y_2	z_4



X	Y	Z
x_2	y_2	z_4

Orden serial equivalente: $t_1; t_3; t_2; t_4$

Read Only Multiversión



ROMV

ROMV es un protocolo híbrido simple que tiene como objetivo reconciliar la simplicidad del (S)2PL convencional o de TimeStamp simple con los beneficios de rendimiento del versionado



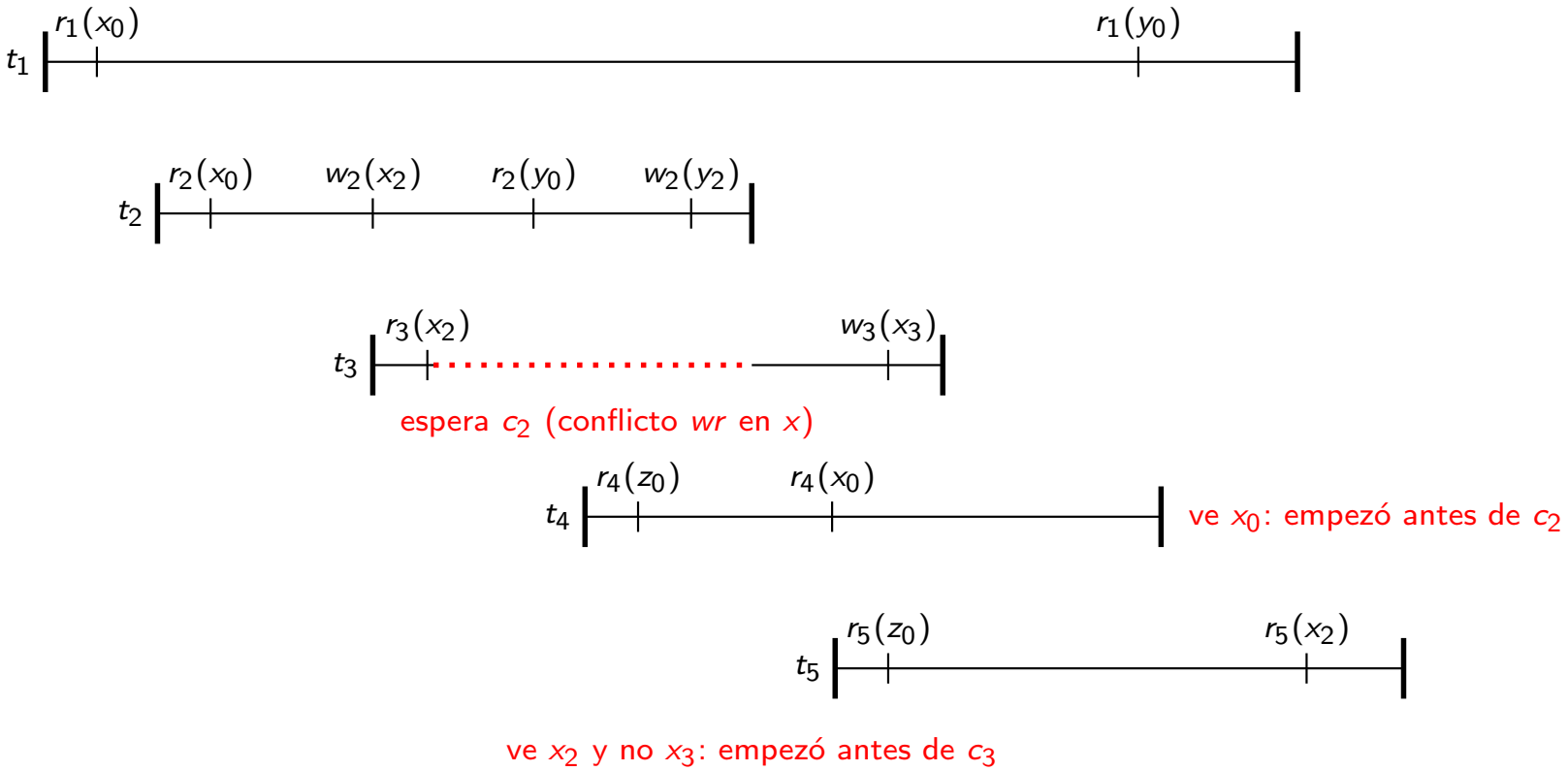
Las transacciones de sólo lectura deben ser marcadas como tales al comenzar.

Read Only Multiversión

- **Las transacciones de actualización** están sujetas al protocolo convencional 2PL. A diferencia del entorno de monoversión convencional, cada paso de escritura **crea** una nueva versión en lugar de sobrescribir; cada versión tiene asignado el *timestamp* de su transacción que *corresponde al tiempo de commit* de la transacción.
- **Las transacciones Read Only** siguen un protocolo similar al MVTO. El timestamp asignado es el de **comienzo de la transacción**. Siempre acceden a la versión con el timestamp más alto que es menor que el suyo, es decir, la última versión commiteada al comenzar.

Los lectores nunca esperan ni abortan: leen versiones ya commiteadas. Las transacciones de actualización no se benefician del versionado.

Ejemplo ROMV



t_1, t_4, t_5: solo lectura (timestamp = comienzo). t_2, t_3: actualización (2PL, timestamp = commit).

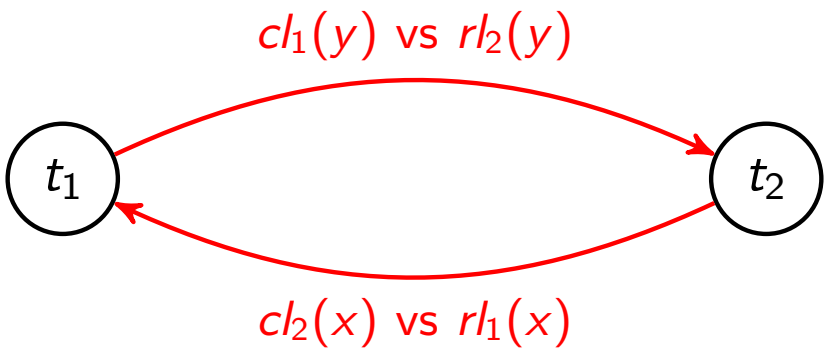
Deadlock en 2V2PL

Tanto 2PL como 2V2PL (y por lo tanto MV2PL) **no evitan** deadlocks

Entrada: $r_1(x)\, r_2(y)\, w_1(y)\, w_2(x)\, c_1\, c_2$

Salida del protocolo:
$rl_1(x)\, r_1(x_0)\, rl_2(y)\, r_2(y_0)\, wl_1(y)\, w_1(y_1)\, wl_2(x)\, w_2(x_2)$

Para commitear, t_1 necesita $cl_1(y)$ y t_2 necesita $cl_2(x)$:



Cada una espera a la otra. El certify lock se pide recién al final, cuando las lecturas ya tienen su *rl* sobre la versión actual.

Bibliografía



Gerhard Weikum and Gottfried Vossen. 2001. *Transactional Information Systems: Theory, Algorithms, and the Practice of Concurrency Control and Recovery*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.